

Graph Theory Lab Manual

Lab 1 Problem 1

Question:

Objective: This practical is designed as a revision of graph concepts previously studied in Discrete Mathematics and Data Structures.

Problem 1: Given the number of vertices and edges of a graph, the task is to represent the adjacency list of a directed graph.

Input: $V=3$, $\text{edges}[][] = \{\{0,1\}, \{1,2\}, \{2,0\}\}$

Output:

0 -> 1

1 -> 2

2 -> 0

Input: $V=4$, $\text{edges}[][] = \{\{0,1\}, \{1,2\}, \{1,3\}, \{2,3\}, \{3,0\}\}$

Output:

0 -> 1

1 -> 2 3

2 -> 3

3 ->

Code:

```
#include <stdio.h>

void main()
{
    int v, e;
    printf("Enter the number of vertices: ");
    scanf("%d", &v);
    printf("Enter the number of edges: ");
    scanf("%d", &e);

    int edges[e][2];
    printf("Enter the edges (source destination):\n");
    for (int i = 0; i < e; i++) {
        scanf("%d %d", &edges[i][0], &edges[i][1]);
    }

    for (int i = 0; i < v; i++) {
        printf("%d -> ", i);
        for (int j = 0; j < e; j++) {
            if (edges[j][0] == i) {
```

```

        printf("%d ", edges[j][1]);
    }
}
printf("\n");
}
}

```

Lab 1 Problem 2

Question:

Problem 2: Write a program to perform the following basic graph-related operations:

1. Represent a graph using adjacency matrix.
2. Represent a graph using adjacency list.
3. Find the degree of each vertex in the graph.
4. Determine whether the graph is simple, complete, or connected.
5. Display total number of vertices and edges.

Input: $V = 5$, edges $[] [] = \{(0, 1), (0, 2), (1, 2), (2, 3), (3, 4)\}$

Output:

1. Adjacency Matrix
2. Adjacency List
3. Degree of each vertex
4. Graph properties
5. Total vertices and edges

Code:

```

#include <stdio.h>

void main()
{
    int v, e;

    printf("Enter the number of vertices: ");
    scanf("%d", &v);

    printf("Enter the number of edges: ");
    scanf("%d", &e);

    int adjMat[v][v];

    for(int i = 0; i < v; i++)
    {
        for(int j = 0; j < v; j++)
        {

```

```

        adjMat[i][j] = 0;
    }
}

printf("Enter the edges (source destination):\n");

for(int i = 0; i < e; i++)
{
    int src, dest;
    scanf("%d %d", &src, &dest);

    adjMat[src][dest]++;
    adjMat[dest][src]++;
}

printf("\nAdjacency Matrix:\n");

for(int i = 0; i < v; i++)
{
    for(int j = 0; j < v; j++)
    {
        printf("%d ", adjMat[i][j]);
    }
    printf("\n");
}

printf("\nAdjacency List:\n");
for(int i = 0; i < v; i++)
{
    printf("%d -> ", i);

    for(int j = 0; j < v; j++)
    {
        if(adjMat[i][j] == 1)
        {
            printf("%d ", j);
        }
    }
    printf("\n");
}

printf("\nDegree of each Vertex:\n");
for(int i = 0; i < v; i++)
{
    int degree = 0;

```

```

        for(int j = 0; j < v; j++)
        {
            degree += adjMat[i][j];
        }

        printf("Vertex %d = %d\n", i, degree);
    }

    int isSimple = 1;
    for(int i = 0; i < v; i++)
    {
        if(adjMat[i][i] != 0)
        {
            isSimple = 0;
            break;
        }
        for(int j = i + 1; j < v; j++)
        {
            if(adjMat[i][j] > 1)
            {
                isSimple = 0;
                break;
            }
        }
        if(!isSimple) break;
    }
    printf("\nSimple Graph    : %s\n", isSimple ? "Yes" : "No");

    int isComplete = isSimple;
    if(isComplete)
    {
        for(int i = 0; i < v; i++)
        {
            int degree = 0;
            for(int j = 0; j < v; j++)
                degree += adjMat[i][j];
            if(degree != v - 1)
            {
                isComplete = 0;
                break;
            }
        }
    }
    printf("Complete Graph : %s\n", isComplete ? "Yes" : "No");

```

```

    int visited[v], que[v];
    int f = 0, r = 0;
    for(int i = 0; i < v; i++) visited[i] = 0;

    visited[0] = 1;
    que[r++] = 0;

    while(f < r)
    {
        int node = que[f++];
        for(int j = 0; j < v; j++)
            if(adjMat[node][j] > 0 && !visited[j])
            {
                visited[j] = 1;
                que[r++] = j;
            }
    }

    int isConnected = 1;
    for(int i = 0; i < v; i++)
        if(!visited[i]) { isConnected = 0; break; }

    printf("Connected Graph: %s\n", isConnected ? "Yes" : "No");

    printf("\nTotal Vertices : %d\n", v);
    printf("Total Edges      : %d\n", e);

}

```

Lab 2

Question:

Objective: To implement Breadth First Search traversal for a graph.

Problem: Perform a Breadth First Search (BFS) traversal starting from vertex 0.

Input: adj[][] = [[1,2], [0,2,3], [0,1,4], [1,4], [2,3]]

Output:

[0, 1, 2, 3, 4]

Code:

```
#include <stdio.h>
```

```
void main()
```

```

{
    int v = 5;
    // Adjacency matrix for:
    // [[1,2], [0,2,3], [0,1,4], [1,4], [2,3]]
    int adjMat[5][5] = {
        {0,1,1,0,0},
        {1,0,1,1,0},
        {1,1,0,0,1},
        {0,1,0,0,1},
        {0,0,1,1,0}
    };
    int visited[5] = {0};
    int que[5];
    int f = 0, r = 0;

    int start = 0;

    visited[start] = 1;
    que[r++] = start;

    printf("BFS Traversal: ");

    while(f < r)
    {
        int current = que[f++];

        printf("%d ", current);

        for(int i = 0; i < v; i++)
        {
            if(adjMat[current][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                que[r++] = i;
            }
        }
    }
}

```

Lab 3

Question:

Objective: To implement Depth First Search traversal for a graph.

Problem: Write a program to perform DFS traversal starting from a given source vertex.

Input: adj[][] = [[1,2], [0,2], [0,1,3,4], [2], [2]]

Output:

[0, 1, 2, 3, 4]

Code:

```
#include <stdio.h>

int adjMat[5][5] = {
    {0,1,1,0,0},
    {1,0,1,0,0},
    {1,1,0,1,1},
    {0,0,1,0,0},
    {0,0,1,0,0}
};

int visited[5] = {0};

void dfs(int current, int v)
{
    visited[current] = 1;
    printf("%d ", current);

    for(int i = 0; i < v; i++)
    {
        if(adjMat[current][i] == 1 && visited[i] == 0)
        {
            dfs(i, v);
        }
    }
}

void main()
{
    int v = 5;
    int start = 0;

    printf("DFS Traversal: ");

    dfs(start, v);
}
```

Lab 4

Question:

Objective: To understand the basic properties and structure of trees using graph operations.

Problem 1: Write a program to perform the following operations on a tree:

1. Identify all pendent (leaf) vertices.
2. Find the degree of each vertex.
3. Find distance between vertices.
4. Find eccentricity of each vertex
5. Find the center of the tree.
6. Verify whether the given graph is a tree or not.

Input: $V = 6$, edges $[[[]] = \{(0,1), (1,2), (1,3), (3,4), (3,5)\}$

Output:

1. List of pendent vertices
2. Degree of each vertex
3. Distance matrix
4. Eccentricity of each vertex
5. Center of the tree
6. Confirmation whether the graph is a tree

Code:

```
#include <stdio.h>

#define V 6

int adj[V][V] = {
    {0,1,0,0,0,0},
    {1,0,1,1,0,0},
    {0,1,0,0,0,0},
    {0,1,0,0,1,1},
    {0,0,0,1,0,0},
    {0,0,0,1,0,0}
};

void bfs(int start, int dist[])
{
    int visited[V] = {0};
    int q[V];
    int f = 0, r = 0;

    for(int i=0;i<V;i++)
        dist[i] = -1;

    visited[start] = 1;
    dist[start] = 0;
    q[r++] = start;

    while(f < r)
```



```

    {
        int curr = q[f++];

        for(int i=0;i<V;i++)
        {
            if(adj[curr][i] == 1 && !visited[i])
            {
                visited[i] = 1;
                dist[i] = dist[curr] + 1;
                q[r++] = i;
            }
        }
    }
}

void main()
{
    int degree[V] = {0};
    int dist[V][V];
    int eccentricity[V];

    printf("1. Pendent (Leaf) Vertices:\n");

    for(int i=0;i<V;i++)
    {
        for(int j=0;j<V;j++)
            degree[i] += adj[i][j];

        if(degree[i] == 1)
            printf("%d ", i);
    }

    printf("\n\n2. Degree of Each Vertex:\n");
    for(int i=0;i<V;i++)
        printf("Vertex %d = %d\n", i, degree[i]);

    printf("\n3. Distance Matrix:\n");

    for(int i=0;i<V;i++)
        bfs(i, dist[i]);

    for(int i=0;i<V;i++)
    {
        for(int j=0;j<V;j++)
            printf("%d ", dist[i][j]);
    }
}

```

```

        printf("\n");
    }

    printf("\n4. Eccentricity of Each Vertex:\n");

    for(int i=0;i<V;i++)
    {
        int max = 0;

        for(int j=0;j<V;j++)
        {
            if(dist[i][j] > max)
                max = dist[i][j];
        }

        eccentricity[i] = max;
        printf("Vertex %d = %d\n", i, max);
    }

    int minEcc = eccentricity[0];

    for(int i=1;i<V;i++)
    {
        if(eccentricity[i] < minEcc)
            minEcc = eccentricity[i];
    }

    printf("\n5. Center of Tree:\n");

    for(int i=0;i<V;i++)
    {
        if(eccentricity[i] == minEcc)
            printf("%d ", i);
    }

    int edges = 0;

    for(int i=0;i<V;i++)
    {
        for(int j=i+1;j<V;j++)
        {
            if(adj[i][j] == 1)
                edges++;
        }
    }

```

```

printf("\n\n6. Tree Verification:\n");

if(edges == V - 1)
    printf("Given graph is a Tree\n");
else
    printf("Given graph is NOT a Tree\n");
}

```

Lab 5 Problem 1

Question:

Objective: To analyze the structural properties of a connected graph using spanning trees, branches, chords, rank, and nullity.

Problem 1: In a social network of N people, some of them are directly connected as friends. If person A is a friend of person B, and person B is a friend of person C, then A, B, and C are all part of the same friend circle (a connected component in graph terms). You are given an undirected graph represented by an adjacency matrix of size $N \times N$, where $\text{matrix}[i][j] = 1$ indicates a direct friendship between person i and person j .

Your task is to determine the total number of friend circles.

Input:

```

4 // total N people
1 1 0 0 // adj. Matrix
1 1 0 0
0 0 1 1
0 0 1 1

```

Output:

```

2

```

Code:

```

#include <stdio.h>

int adj[4][4] = {
    {1,1,0,0},
    {1,1,0,0},
    {0,0,1,1},
    {0,0,1,1}
};

int visited[4] = {0};

void dfs(int v, int n)
{
    visited[v] = 1;
}

```

```

        for(int i = 0; i < n; i++)
        {
            if(adj[v][i] == 1 && visited[i] == 0)
            {
                dfs(i, n);
            }
        }
    }

void main()
{
    int n = 4;
    int friendCircles = 0;

    for(int i = 0; i < n; i++)
    {
        if(visited[i] == 0)
        {
            dfs(i, n);
            friendCircles++;
        }
    }

    printf("Total Friend Circles = %d", friendCircles);
}

```

Lab 5 Problem 2

Question:

Problem 2: Write a program to perform the following operations on a connected graph:

1. Construct a spanning tree of the graph.
2. Identify all branches and chords.
3. Find all fundamental circuits
4. Calculate: Rank and Nullity of the graph
5. Verify that: Number of branches = Rank and Number of chords = Nullity.

Input:

Enter number of vertices: 5

Enter number of edges: 6

edges[][] = {(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)}

Output:

Branches:

(0,1)

(0,2)

(1,3)

(2,4)

Chords:

(1,2)

(3,4)

Fundamental Circuits: C1 = {(0,1),(0,2),(1,2)} C2 = {(1,3),(2,4),(3,4)}

Rank of Graph = 4

Nullity of Graph = 2

Code:

```
#include <stdio.h>
```

```
void print_edge(int u, int v) {  
    if (u < v) {  
        printf("(%d,%d)", u, v);  
    } else {  
        printf("(%d,%d)", v, u);  
    }  
}
```

```
int main() {  
    int V = 5;  
    int E = 6;  
    int edges[6][2] = {{0,1}, {0,2}, {1,2}, {1,3}, {2,4}, {3,4}};  
  
    int branches[10][2];  
    int num_branches = 0;  
  
    int chords[10][2];  
    int num_chords = 0;  
  
    int parent[10];  
    for (int i = 0; i < V; i++) {  
        parent[i] = i;  
    }  
  
    int tree_adj[10][10] = {0};  
  
    for (int i = 0; i < E; i++) {  
        int u = edges[i][0];  
        int v = edges[i][1];  
  
        int root_u = u;  
        while (parent[root_u] != root_u) {
```

```

        root_u = parent[root_u];
    }

    int root_v = v;
    while (parent[root_v] != root_v) {
        root_v = parent[root_v];
    }

    if (root_u != root_v) {
        parent[root_u] = root_v;

        branches[num_branches][0] = u;
        branches[num_branches][1] = v;
        num_branches++;

        tree_adj[u][v] = 1;
        tree_adj[v][u] = 1;
    } else {
        chords[num_chords][0] = u;
        chords[num_chords][1] = v;
        num_chords++;
    }
}

printf("Output:\nBranches:\n");
for (int i = 0; i < num_branches; i++) {
    printf("(%d,%d)\n", branches[i][0], branches[i][1]);
}

printf("\nChords:\n");
for (int i = 0; i < num_chords; i++) {
    printf("(%d,%d)\n", chords[i][0], chords[i][1]);
}

printf("\nFundamental Circuits:\n");
for (int c = 0; c < num_chords; c++) {
    int start = chords[c][0];
    int end = chords[c][1];

    int stack[20];
    int top = 0;
    int visited[10] = {0};
    int parent_node[10];

    for(int i = 0; i < V; i++) parent_node[i] = -1;

```

```

    stack[top++] = start;
    visited[start] = 1;

    while (top > 0) {
        int curr = stack[--top];

        if (curr == end) break;

        for (int i = 0; i < V; i++) {
            if (tree_adj[curr][i] == 1 && visited[i] == 0) {
                visited[i] = 1;
                parent_node[i] = curr;
                stack[top++] = i;
            }
        }
    }

    printf("C%d = {", c + 1);

    int curr = end;
    int is_first = 1;
    while (curr != start) {
        int p = parent_node[curr];
        if (!is_first) printf(",");
        print_edge(p, curr);
        curr = p;
        is_first = 0;
    }

    printf(",");
    print_edge(start, end);
    printf("}\n");
}

int rank = V - 1;
int nullity = E - V + 1;

printf("\nRank of Graph = %d\n", rank);
printf("Nullity of Graph = %d\n", nullity);

printf("\n--- Verification ---\n");
printf("Number of branches (%d) == Rank (%d) -> %s\n",
num_branches, rank, (num_branches == rank) ? "Verified" :
"Failed");

```

```

    printf("Number of chords (%d) == Nullity (%d) -> %s\n",
num_chords, nullity, (num_chords == nullity) ? "Verified" :
"Failed");

    return 0;
}

```

Lab 6

Question:

Objective: To identify articulation points in an undirected graph and determine connectivity and separability of a graph.

Problem: Write a program to find all cut-vertices of a graph.

Input: Connected graph with vertices and edges.

Output:

List of articulation points.

Code:

```

#include <stdio.h>

#define MAX 10

int adj[MAX][MAX];
int visited[MAX];
int V, E;

int dfs_count(int src) {
    visited[src] = 1;
    int count = 1;
    for (int i = 0; i < V; i++)
        if (adj[src][i] && !visited[i])
            count += dfs_count(i);
    return count;
}

int main() {
    int u, v, i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter number of edges: ");
    scanf("%d", &E);

    for (i = 0; i < V; i++)

```



```

        for (j = 0; j < V; j++)
            adj[i][j] = 0;

    printf("Enter edges (u v):\n");
    for (i = 0; i < E; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = adj[v][u] = 1;
    }

    printf("\nArticulation Points (Cut Vertices):\n");
    int found = 0;

    for (int skip = 0; skip < V; skip++) {

        for (i = 0; i < V; i++)
            visited[i] = 0;

        visited[skip] = 1;

        int start = (skip == 0) ? 1 : 0;
        int reachable = dfs_count(start);

        if (reachable < V - 1) {
            printf("  Vertex %d is an Articulation Point\n",
skip);
            found = 1;
        }
    }

    if (!found)
        printf("  No articulation points found.\n");

    return 0;
}

```

Lab 7

Question:

Objective: Determine connectivity and separability of a graph

Problem: Write a program to:

1. Find vertex connectivity
2. Find edge connectivity
3. Construct fundamental cut-sets
4. Determine whether graph is separable or non-separable

Input: Undirected graph.

Output: Connectivity values and graph type

Code:

```
#include <stdio.h>

int V = 4;
int adj[4][4] = {
    {0, 1, 1, 0},
    {1, 0, 1, 0},
    {1, 1, 0, 1},
    {0, 0, 1, 0}
};

void dfs(int u, int visited[], int skip_v) {
    visited[u] = 1;
    for (int v = 0; v < V; v++) {
        if (adj[u][v] && !visited[v] && v != skip_v) {
            dfs(v, visited, skip_v);
        }
    }
}

int is_connected(int skip_v, int skip_u_edge, int skip_v_edge) {
    int visited[10] = {0};
    int start = (skip_v == 0) ? 1 : 0;
    if (start >= V) return 0;

    int temp = 0;
    if (skip_u_edge != -1 && skip_v_edge != -1) {
        temp = adj[skip_u_edge][skip_v_edge];
        adj[skip_u_edge][skip_v_edge] = 0;
        adj[skip_v_edge][skip_u_edge] = 0;
    }

    dfs(start, visited, skip_v);

    if (skip_u_edge != -1 && skip_v_edge != -1) {
        adj[skip_u_edge][skip_v_edge] = temp;
        adj[skip_v_edge][skip_u_edge] = temp;
    }

    for (int i = 0; i < V; i++) {
        if (i != skip_v && !visited[i]) return 0;
    }
}
```

```

        return 1;
    }

int main() {
    int vc = V - 1;
    for (int i = 0; i < V; i++) {
        if (!is_connected(i, -1, -1)) {
            vc = 1;
            break;
        }
    }

    int ec = V - 1;
    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            if (adj[i][j]) {
                if (!is_connected(-1, i, j)) {
                    ec = 1;
                    break;
                }
            }
        }
        if (ec == 1) break;
    }

    printf("Vertex Connectivity: %d\n", vc);
    printf("Edge Connectivity: %d\n", ec);

    if (vc == 1) {
        printf("Graph is Separable\n");
    } else {
        printf("Graph is Non-separable\n");
    }

    return 0;
}

```

Lab 8

Question:

Objective: To determine the dominating set and domination number of a graph.

Problem: Write a program to:

1. Find dominating sets
2. Find domination number
3. Find total dominating sets

4. Find total domination number

Input: N = 4, M = 4, Edges = {(1,2), (1,3), (3,4), (2,4)}

Output:

Possible Dominating Sets:

{1,4}

{2,3}

{1,2}

{1,3}

{2,4}

{3,4}

Domination Number = 2

Total Dominating Sets: {1,3} {2,4}

Total Domination Number = 2

Code:

```
#include <stdio.h>

int main() {
    int N = 4;
    int edges[4][2] = {{0, 1}, {0, 2}, {2, 3}, {1, 3}};
    int adj[4][4] = {0};

    for (int i = 0; i < 4; i++) {
        adj[edges[i][0]][edges[i][1]] = 1;
        adj[edges[i][1]][edges[i][0]] = 1;
    }

    int min_dom = N + 1;
    int min_tdom = N + 1;

    printf("Possible Dominating Sets:\n");
    for (int i = 1; i < (1 << N); i++) {
        int is_dom = 1;
        int is_tdom = 1;
        int size = 0;

        for (int v = 0; v < N; v++) {
            if (i & (1 << v)) size++;
        }

        for (int v = 0; v < N; v++) {
            int has_neighbor_in_set = 0;
            for (int u = 0; u < N; u++) {
                if ((i & (1 << u)) && adj[v][u]) {
                    has_neighbor_in_set = 1;
                }
            }
        }
    }
}
```

```

        break;
    }
}

if (!(i & (1 << v)) && !has_neighbor_in_set) {
    is_dom = 0;
}
if (!has_neighbor_in_set) {
    is_tdom = 0;
}
}

if (is_dom) {
    printf("{");
    int first = 1;
    for (int v = 0; v < N; v++) {
        if (i & (1 << v)) {
            if (!first) printf(",");
            printf("%d", v + 1);
            first = 0;
        }
    }
    printf("}\n");
    if (size < min_dom) min_dom = size;
}

printf("Domination Number = %d\n\n", min_dom);

printf("Total Dominating Sets:\n");
for (int i = 1; i < (1 << N); i++) {
    int is_tdom = 1;
    int size = 0;

    for (int v = 0; v < N; v++) {
        if (i & (1 << v)) size++;
    }

    for (int v = 0; v < N; v++) {
        int has_neighbor_in_set = 0;
        for (int u = 0; u < N; u++) {
            if ((i & (1 << u)) && adj[v][u]) {
                has_neighbor_in_set = 1;
                break;
            }
        }
    }
}

```

```

        if (!has_neighbor_in_set) {
            is_tdom = 0;
            break;
        }
    }

    if (is_tdom) {
        printf("{");
        int first = 1;
        for (int v = 0; v < N; v++) {
            if (i & (1 << v)) {
                if (!first) printf(",");
                printf("%d", v + 1);
                first = 0;
            }
        }
        printf("}\n");
        if (size < min_tdom) min_tdom = size;
    }
}

printf("Total Domination Number = %d\n", min_tdom);

return 0;
}

```

Lab 9

Question:

Objective: To determine the total dominating set and total domination number of a graph.

Problem: Write a program to:

1. Find total dominating sets
2. Find total domination number

Input: $N = 4$, $M = 4$, Edges = $\{(1,2), (1,3), (3,4), (2,4)\}$

Output:

Total Dominating Sets: $\{1,3\} \{2,4\}$

Total Domination Number = 2

Code:

```

#include <stdio.h>

int main() {
    int N = 4;
    int edges[4][2] = {{0, 1}, {0, 2}, {2, 3}, {1, 3}};
    int adj[4][4] = {0};
}

```

```

for (int i = 0; i < 4; i++) {
    adj[edges[i][0]][edges[i][1]] = 1;
    adj[edges[i][1]][edges[i][0]] = 1;
}

int min_tdom = N + 1;

printf("Total Dominating Sets:\n");
for (int i = 1; i < (1 << N); i++) {
    int is_tdom = 1;
    int size = 0;

    for (int v = 0; v < N; v++) {
        if (i & (1 << v)) size++;
    }

    for (int v = 0; v < N; v++) {
        int has_neighbor_in_set = 0;
        for (int u = 0; u < N; u++) {
            if ((i & (1 << u)) && adj[v][u]) {
                has_neighbor_in_set = 1;
                break;
            }
        }
        if (!has_neighbor_in_set) {
            is_tdom = 0;
            break;
        }
    }

    if (is_tdom) {
        printf("{");
        int first = 1;
        for (int v = 0; v < N; v++) {
            if (i & (1 << v)) {
                if (!first) printf(",");
                printf("%d", v + 1);
                first = 0;
            }
        }
        printf("}\n");
        if (size < min_tdom) min_tdom = size;
    }
}

printf("Total Domination Number = %d\n", min_tdom);

```

```
        return 0;
    }
}
```

Lab 10

Question:

Objective: To determine different types of matching in graphs

Problem: Write a program to determine: Maximal matching, Maximum matching, and Perfect matching

Input: Bipartite graph.

Output:

Matching sets and type of matching.

Code:

```
#include <stdio.h>

int M = 3;
int N = 3;
int bpGraph[3][3] = {
    {1, 0, 0},
    {1, 1, 0},
    {0, 0, 1}
};

int bpm(int u, int matchR[], int seen[]) {
    for (int v = 0; v < N; v++) {
        if (bpGraph[u][v] && !seen[v]) {
            seen[v] = 1;
            if (matchR[v] < 0 || bpm(matchR[v], matchR, seen)) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    int matchR[3];
    for (int i = 0; i < N; i++) matchR[i] = -1;

    int result = 0;
    for (int u = 0; u < M; u++) {
        int seen[3] = {0};
```



```

        if (bpm(u, matchR, seen)) {
            result++;
        }
    }

    printf("Maximum Matching Edges:\n");
    for (int i = 0; i < N; i++) {
        if (matchR[i] >= 0) {
            printf("(%d, %d)\n", matchR[i], i + M);
        }
    }
    printf("Maximum Matching Size: %d\n", result);

    printf("\nMaximal Matching Edges (Greedy Approach):\n");
    int max_match[3] = {-1, -1, -1};
    int x_matched[3] = {0};

    for (int u = 0; u < M; u++) {
        for (int v = 0; v < N; v++) {
            if (bpGraph[u][v] && max_match[v] == -1 && !
x_matched[u]) {
                max_match[v] = u;
                x_matched[u] = 1;
                printf("(%d, %d)\n", u, v + M);
                break;
            }
        }
    }

    if (result == M && result == N) {
        printf("\nType: Perfect Matching Achieved\n");
    } else {
        printf("\nType: Perfect Matching Not Achieved\n");
    }

    return 0;
}

```

Lab 11

Question:

Objective: Verify Hall's Condition

Problem: Write a program to check whether Hall's condition holds for a given bipartite graph.

Input: Bipartite graph with sets X and Y.

Output:

Hall's condition satisfied or not satisfied.

Code:

```
#include <stdio.h>

int main() {
    int nX = 3;
    int nY = 3;

    int adj[3][3] = {
        {1, 1, 0},
        {1, 0, 0},
        {0, 1, 0}
    };

    int satisfies_hall = 1;
    int total_subsets = 1 << nX;

    for (int i = 1; i < total_subsets; i++) {
        int size_W = 0;
        int size_NW = 0;
        int neighbors[10] = {0};

        for (int j = 0; j < nX; j++) {
            if (i & (1 << j)) {
                size_W++;
                for (int k = 0; k < nY; k++) {
                    if (adj[j][k] == 1) {
                        neighbors[k] = 1;
                    }
                }
            }
        }

        for (int k = 0; k < nY; k++) {
            if (neighbors[k] == 1) {
                size_NW++;
            }
        }

        if (size_NW < size_W) {
            satisfies_hall = 0;
            break;
        }
    }
}
```

```

    }

    if (satisfies_hall) {
        printf("Hall's condition satisfied\n");
    } else {
        printf("Hall's condition not satisfied\n");
    }

    return 0;
}

```

Lab 12

Question:

Objective: To construct different matrices associated with graphs.

Problem: Write a program to construct: Adjacency Matrix, Degree Matrix, and Laplacian Matrix

Input: $N = 5$, $M = 4$, Edges = $\{(1,2), (2,3), (4,5), (1,5)\}$

Output

Adjacency, Degree, and Laplacian matrices.

Code:

```

#include <stdio.h>

int main() {
    int N = 5;
    int edges[4][2] = {{1, 2}, {2, 3}, {4, 5}, {1, 5}};
    int num_edges = 4;

    int A[5][5] = {0};
    int D[5][5] = {0};
    int L[5][5] = {0};

    for (int i = 0; i < num_edges; i++) {
        int u = edges[i][0] - 1;
        int v = edges[i][1] - 1;

        A[u][v] = 1;
        A[v][u] = 1;

        D[u][u]++;
        D[v][v]++;
    }

    for (int i = 0; i < N; i++) {

```

```

        for (int j = 0; j < N; j++) {
            L[i][j] = D[i][j] - A[i][j];
        }
    }

    printf("Adjacency Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", A[i][j]);
        }
        printf("\n");
    }

    printf("\nDegree Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", D[i][j]);
        }
        printf("\n");
    }

    printf("\nLaplacian Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%2d ", L[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Lab 13

Question:

Objective: To compute eigenvalues and eigenvectors of graph matrices.

Problem: Write a program to calculate eigenvalues and eigenvectors of adjacency and Laplacian matrix.

Input: Adjacency or Laplacian matrix.

Output:

Eigenvalues and corresponding eigenvectors.

Code:

```

#include <stdio.h>
#include <math.h>

```

```

#define N 5

void multiply(double A[N][N], double v[N], double result[N]) {
    for (int i = 0; i < N; i++) {
        result[i] = 0;
        for (int j = 0; j < N; j++)
            result[i] += A[i][j] * v[j];
    }
}

double norm(double v[N]) {
    double s = 0;
    for (int i = 0; i < N; i++) s += v[i] * v[i];
    return sqrt(s);
}

double powerIteration(double A[N][N], double eigenvector[N]) {
    double v[N], av[N];
    for (int i = 0; i < N; i++) v[i] = 1.0;

    for (int iter = 0; iter < 1000; iter++) {
        multiply(A, v, av);
        double n = norm(av);
        for (int i = 0; i < N; i++) v[i] = av[i] / n;
    }

    multiply(A, v, av);
    double eigenvalue = 0;
    for (int i = 0; i < N; i++) eigenvalue += v[i] * av[i];
    for (int i = 0; i < N; i++) eigenvector[i] = v[i];
    return eigenvalue;
}

int main() {
    double A[N][N] = {
        {0,1,0,0,1},
        {1,0,1,0,0},
        {0,1,0,1,0},
        {0,0,1,0,1},
        {1,0,0,1,0}
    };

    double degree[N];
    double L[N][N] = {0};

```

```

    for (int i = 0; i < N; i++) {
        degree[i] = 0;
        for (int j = 0; j < N; j++) degree[i] += A[i][j];
    }
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            L[i][j] = (i == j ? degree[i] : 0) - A[i][j];

    printf("Adjacency Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) printf("%.0f ", A[i][j]);
        printf("\n");
    }

    printf("\nLaplacian Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) printf("%.0f ", L[i][j]);
        printf("\n");
    }

    double eigvec[N];
    double eigval = powerIteration(A, eigvec);

    printf("\nDominant Eigenvalue (Adjacency): %.4f\n", eigval);
    printf("Corresponding Eigenvector: ");
    for (int i = 0; i < N; i++) printf("%.4f ", eigvec[i]);
    printf("\n");

    double eigval_l = powerIteration(L, eigvec);
    printf("\nDominant Eigenvalue (Laplacian): %.4f\n",
eigval_l);
    printf("Corresponding Eigenvector: ");
    for (int i = 0; i < N; i++) printf("%.4f ", eigvec[i]);
    printf("\n");

    return 0;
}

```

Lab 14

Question:

Objective: To perform spectral clustering on a graph using the Laplacian matrix and eigenvectors.

Problem: Write a program to perform spectral clustering on an undirected graph by using

the graph Laplacian matrix.

The program should:

1. Construct the adjacency matrix of the graph.
2. Compute the degree matrix.
3. Construct the Laplacian matrix.
4. Compute eigenvalues and eigenvectors of the Laplacian matrix.
5. Partition the graph into clusters using spectral properties.

Input:

N = 6

Edges = {(0,1),(0,2),(1,2),(3,4),(4,5),(3,5)}

Output:

Cluster 1: {0,1,2}

Cluster 2: {3,4,5}

Code:

```
#include <stdio.h>
#include <math.h>

#define N 6

void multiply(double A[N][N], double v[N], double result[N]) {
    for (int i = 0; i < N; i++) {
        result[i] = 0;
        for (int j = 0; j < N; j++)
            result[i] += A[i][j] * v[j];
    }
}

double dot(double a[N], double b[N]) {
    double s = 0;
    for (int i = 0; i < N; i++) s += a[i] * b[i];
    return s;
}

double norm(double v[N]) {
    return sqrt(dot(v, v));
}

void getFiedler(double L[N][N], double fiedler[N]) {
    double v[N], av[N];
    for (int i = 0; i < N; i++) v[i] = (i % 2 == 0) ? 1.0 : -1.0;

    double n = norm(v);
    for (int i = 0; i < N; i++) v[i] /= n;
```

```

double shift = 10.0;
double shifted[N][N];
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        shifted[i][j] = L[i][j] + (i == j ? shift : 0);

for (int iter = 0; iter < 2000; iter++) {
    multiply(shifted, v, av);
    double n2 = norm(av);
    for (int i = 0; i < N; i++) v[i] = av[i] / n2;
}

multiply(L, v, av);
double lambda = dot(v, av);

for (int i = 0; i < N; i++) fiedler[i] = v[i];
}

int main() {
    int edges[][2] = {{0,1},{0,2},{1,2},{3,4},{4,5},{3,5}};
    int m = 6;

    double A[N][N] = {0};
    for (int i = 0; i < m; i++) {
        A[edges[i][0]][edges[i][1]] = 1;
        A[edges[i][1]][edges[i][0]] = 1;
    }

    double L[N][N] = {0};
    for (int i = 0; i < N; i++) {
        double deg = 0;
        for (int j = 0; j < N; j++) deg += A[i][j];
        for (int j = 0; j < N; j++)
            L[i][j] = (i == j ? deg : 0) - A[i][j];
    }

    double fiedler[N];
    getFiedler(L, fiedler);

    printf("Fiedler Vector: ");
    for (int i = 0; i < N; i++) printf("%.4f ", fiedler[i]);
    printf("\n");

    printf("Cluster 1: ");
    for (int i = 0; i < N; i++) if (fiedler[i] < 0) printf("%d ",

```



```

i);
    printf("\n");

    printf("Cluster 2: ");
    for (int i = 0; i < N; i++) if (fiedler[i] >= 0) printf("%d
", i);
    printf("\n");

    return 0;
}

```

Lab 15

Question:

Objective: To find shortest paths between all pairs of vertices.

Problem: Write a program to implement Floyd–Warshall Algorithm.

Input:

Weighted graph represented using adjacency matrix.

Output:

Shortest path matrix between all pairs of vertices.

Code:

```

#include <stdio.h>

#define INF 99999
#define V 4

int main() {
    int graph[V][V] = {
        {0, 5, INF, 10},
        {INF, 0, 3, INF},
        {INF, INF, 0, 1},
        {INF, INF, INF, 0}
    };

    int dist[V][V];

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            dist[i][j] = graph[i][j];
        }
    }

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {

```

```

        for (int j = 0; j < V; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

printf("Shortest path matrix between all pairs of vertices:
\n");
for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        if (dist[i][j] == INF) {
            printf("INF ");
        } else {
            printf("%3d ", dist[i][j]);
        }
    }
    printf("\n");
}

return 0;
}

```